

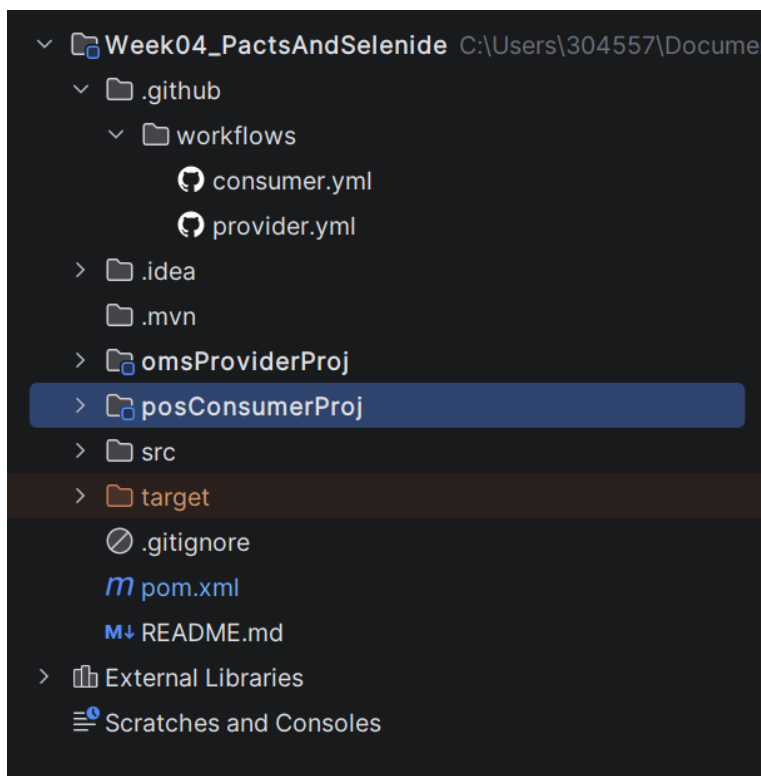
Week 4 Gate Report

1. What Is This Project?

This project is called Week04_PactsAndSelenide. It has two main parts:

- Pact testing - this checks that two services (a POS app and an OMS app) can talk to each other correctly.
- Selenide testing - this checks that a web page works correctly by opening it in a browser and clicking around, automatically.

Both parts use Java and Maven. Maven is the tool that downloads libraries and runs the tests.



As you can see above, the project has two sub-projects inside it:

- omsProviderProj - this is the "provider". It is the service that receives requests (like an order management system).
- posConsumerProj - this is the "consumer". It is the service that sends requests (like a point-of-sale app).

There is also a .github/workflows folder. This holds files that tell GitHub how to automatically run the tests whenever code is pushed. This is called CI, or "Continuous Integration".

2. What Is Pact Testing?

Normally, to test if two services work together, you would have to run both of them at the same time. That can be slow and complicated.

Pact solves this in a clever way. Instead of running both services together, it does this:

1. The consumer (POS app) writes a test that says: "If I send this request, I expect this response back."
2. This expectation is saved into a file. This file is called a contract (or a "pact").
3. The contract is uploaded to a small server called the Pact Broker.
4. The provider (OMS app) downloads that contract from the broker and checks: "Can I actually give that exact response?"

If both sides agree, the test passes. If they don't agree, you find out immediately - without needing to run the whole system together.

2.1 The Consumer Side (posConsumerProj)

The consumer side is where we pretend to be the POS app and describe what we expect from the OMS service.

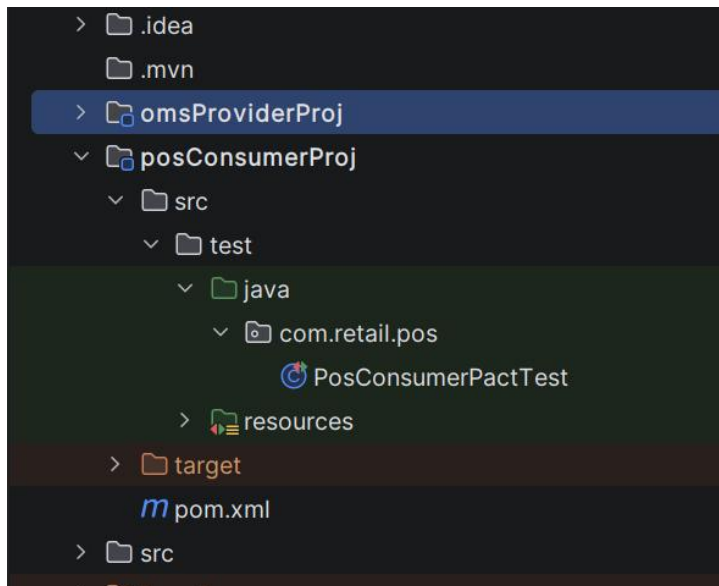
The file `PosConsumerPactTest.java` has several tests. Each test describes one interaction. For example:

- `getOrder` - asks for order 123 and expects a status of `CONFIRMED`.
- `createOrder` - sends a new order for SKU-9 and expects a status 201 (created).
- `updateOrderStatus` - changes order 123's status to `SHIPPED`.
- `getInventory` - asks how much stock SKU-9 has.
- `cancelOrder` - cancels order 123 and expects a 204 (no content) response.

Take the `getOrder` test as an example. It is written using a builder pattern, where one method call is chained after another to describe the interaction step by step: it states the precondition ("order 123 exists"), the request being made (a GET to `/orders/123`), and the expected response (status 200, with an id, a status, and a total in the body).

In plain English, this says: "Given order 123 exists, when I do a GET request to `/orders/123`, I expect back a 200 response with an id, a status, and a total."

When this test runs, Pact creates a fake (mock) server that behaves exactly like this. The test then sends a real HTTP request to that fake server, just to prove the request and response shapes match.



2.2 The Provider Side (omsProviderProj)

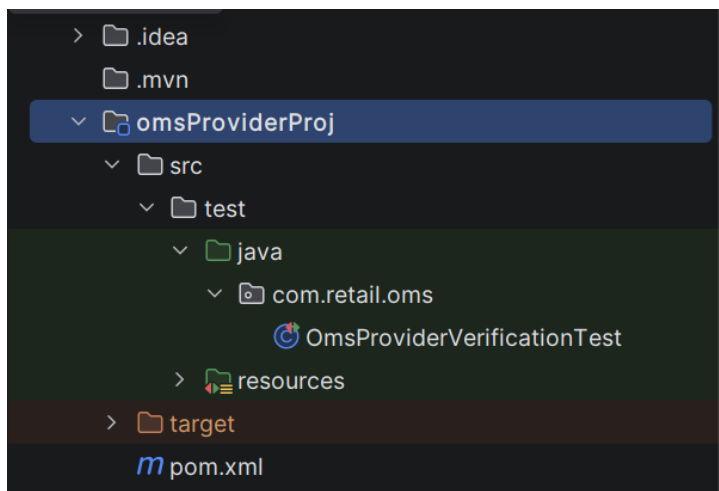
The provider side is where we pretend to be the OMS service and prove that we can actually produce the responses the consumer expects.

The file `OmsProviderVerificationTest.java` does this. Instead of running the real OMS service, it uses a tool called WireMock to fake the responses. This keeps the test simple and fast.

Each contract has a "state", like "order 123 exists". The provider test has a matching method for each state, which sets up the fake response.

For the "order 123 exists" state, the matching method tells WireMock: whenever a GET request comes in for `/orders/123`, respond with a 200 status, a JSON content type header, and a body containing the id, status (CONFIRMED), and total for that order.

This means: "When the state is order 123 exists, and someone calls GET `/orders/123`, respond with this JSON." Pact then checks that this response matches what the consumer expected.



2.3 The Pact Broker

The Pact Broker is a small web server that stores all the contracts. Think of it like a shared mailbox:

- The consumer drops its contract (expectations) into the mailbox.
- The provider checks the mailbox and verifies it can meet those expectations.
- The broker also keeps track of which versions of each service have been verified, so the team can later ask: "Is it safe to deploy this version?"

In this project, the broker runs using Docker, with a Postgres database behind it to store the data.

3. Can I Deploy?

Pact has a feature called "can-i-deploy". After both the consumer and the provider tests pass, this command checks the broker and answers a simple question:

"If I deploy this version of the consumer, will it still work correctly with whatever version of the provider is currently live?"

If the answer is yes, the deployment is considered safe. If the answer is no, the deployment is blocked. This is a safety net that catches mismatches before they reach production.

4. Running Tests Automatically with GitHub Actions

Instead of running these tests by hand every time, this project uses GitHub Actions to run them automatically. There are two workflow files:

- consumer.yml - runs whenever code changes inside posConsumerProj.
- provider.yml - runs whenever code changes inside posConsumerProj or omsProviderProj.

4.1 What the Consumer Workflow Does

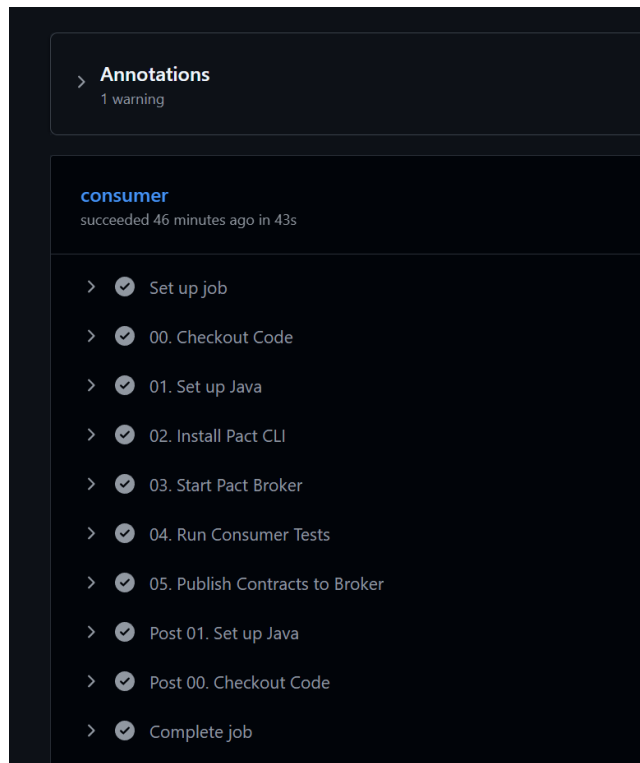
The consumer.yml workflow does the following steps, in order:

1. Checks out the code from GitHub.
2. Sets up Java version 22.
3. Installs the Pact command-line tool.
4. Starts the Pact Broker using Docker.
5. Runs the consumer tests (PosConsumerPactTest).
6. Publishes the resulting contract files to the Pact Broker.

4.2 What the Provider Workflow Does

The provider.yml workflow does the following steps, in order:

1. Checks out the code from GitHub.
2. Sets up Java version 22.
3. Installs the Pact command-line tool.
4. Starts the Pact Broker using Docker.
5. Runs the provider verification test against WireMock.
6. Runs the verification again, this time recording results to the broker.
7. Records that this version of the provider has been deployed.
8. Runs "can-i-deploy" to confirm the consumer is still safe to deploy.



Each step in the picture above matches one line in the workflow file. When every step has a checkmark, it means the whole process worked: the tests ran, and the contract was published to the broker.

5. What Is Selenide Testing?

The second part of this project is about testing a website using a browser, automatically. The tool used for this is called Selenide. Selenide makes it easier to write browser tests than plain Selenium.

The idea is simple: instead of a person clicking buttons and typing into boxes, the test code does it for you, and then checks if the result is correct.

This project uses a pattern called the Page Object pattern. This means we create one class for each page of the website, and that class knows how to find and interact with the elements on that page. There are three files involved: `catalogPage`, `productPage`, and `SelenideProjTest`.

5.1 The Page Object: `catalogPage`

`catalogPage` represents the catalog (product listing) page. It has:

- A search box, found using its `data-test` attribute.
- A list of product cards, also found using their `data-test` attribute.

It has a few methods:

- `openPage` - opens the catalog page in the browser, at `http://localhost:5173/catalog`.
- `searchFor` - waits until the search box is visible, types in a keyword, then presses Enter.
- `cards` - returns the list of product cards currently shown on the page.
- `click_product` - clicks the link inside the first product card, and then returns a `productPage` object, since clicking a product takes you to a new page.

In plain English: "Open the catalog page. Type a keyword into the search box and press Enter. If needed, click into a product, which takes you to that product's page."

5.2 The Page Object: `productPage`

`productPage` represents the page you land on after clicking a product. It has one element:

- `availabilityBadge` - a small badge on the page, found using its `data-testid` attribute, that shows whether the product is in stock or not.

It has one method, `availabilityBadge`, which simply returns that badge element so a test can read its text.

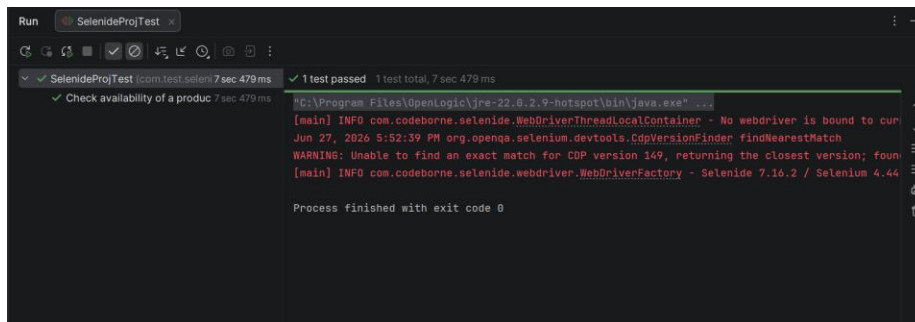
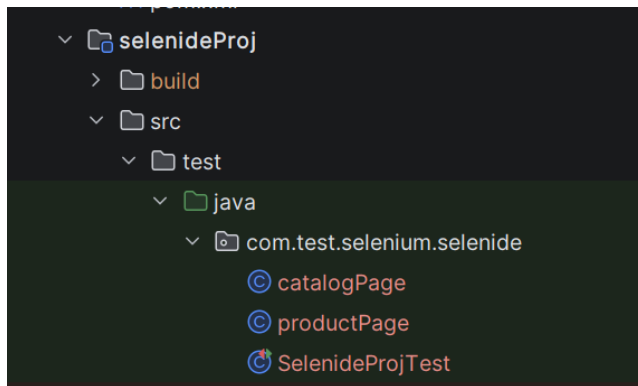
5.3 The Test: `SelenideProjTest`

This test is called "Check availability of a product". It does the following, in order:

1. Opens the catalog page using `catalogPage`, then searches for the keyword "headphone".
2. Clicks on the first matching product, which moves to the `productPage`.
3. Reads the text inside the availability badge on that product page.
4. Checks that the badge's text contains the words "In stock".

In plain English: "Open the catalog, search for headphones, open the first result, and make sure it shows as in stock." If the badge says something else, like "Out of stock", the test fails.

This shows how Selenide tests can move across more than one page: the test starts on the catalog page, moves to a product page, and checks something on that second page, all in one flow.



Picture 6: The test run in IntelliJ. It shows 1 test passed, confirming the product was found and is in stock.

Using Page Objects like this keeps tests easy to read. If the website's layout changes later, you only need to update `catalogPage` or `productPage`, not the test itself.

6. Project Setup Details (pom.xml)

Each sub-project has its own `pom.xml` file. This file tells Maven which Java version to use and which libraries (dependencies) to download. Here are the important settings for `posConsumerProj`:

- Java version: 22 (set using `maven.compiler.source` and `maven.compiler.target`).
- File encoding: UTF-8.
- Pact version: 4.6.20 - this is the version of the Pact library used for consumer and provider testing.
- WireMock version: 3.13.2 - used on the provider side to fake HTTP responses.
- Pact Broker URL: `http://localhost:9292` - this is the address Maven uses to publish and look up contracts.

The `pom.xml` also lists the test libraries this project depends on:

- `rest-assured` - used to send HTTP requests and check responses in tests.
- `junit-jupiter-api` and `junit-jupiter-engine` - the JUnit 5 testing framework that runs the tests.
- `json-schema-validator` (both the `rest-assured` and `networknt` versions) - used to check that JSON responses match an expected schema.

- wiremock - fakes HTTP responses for provider testing.
- pact-jvm consumer and provider junit5 libraries - the core Pact testing tools.

Finally, the build section configures two plugins:

- maven-surefire-plugin - this runs the tests, and passes along settings like where to save contract files and whether to publish results to the broker.
- pact-provider-maven-plugin - this is used to publish contracts to the Pact Broker, tagging them with a version, branch name, and a tag (week4).

7. How to Run This Project Locally

These are the steps to set up and run the project on your own computer, in the right order.

7.1 Before You Start

- Install Java 22 (the JDK).
- Install Maven.
- Install Docker (this is needed to run the Pact Broker).
- Open the project folder in IntelliJ (or any Java IDE).

7.2 Step 1: Start the Pact Broker

The Pact Broker needs to be running before you publish or verify any contracts. It is started using Docker Compose. From the project folder, run:

```
docker compose -f posConsumerProj/src/test/resources/docker-compose.pact-broker.yml up -d --wait
```

This starts both the Postgres database and the Pact Broker itself, and waits until they are healthy before continuing. Once it is running, you can open <http://localhost:9292> in a browser to see the broker's web page.

To stop and clean up the broker later, run:

```
docker compose -f posConsumerProj/src/test/resources/docker-compose.pact-broker.yml down -v
```

7.3 Step 2: Run the Consumer Tests

From the project's root folder, move into posConsumerProj and run the consumer Pact test:

```
cd posConsumerProj
```

```
mvn -Dtest=PosConsumerPactTest test
```

This runs all the consumer tests and creates contract files inside the target/pacts folder.

7.4 Step 3: Publish the Contracts to the Broker

Still inside `posConsumerProj`, run the Pact provider Maven plugin's publish goal to upload the contract files to the broker:

```
mvn pact:publish
```

After this runs, you should be able to see the new contract appear on the Pact Broker's web page.

7.5 Step 4: Run the Provider Verification

Now move into the provider project and run the verification test. This test starts WireMock, sets up the fake responses for each state, and checks that the contracts from the broker are satisfied:

```
cd ../omsProviderProj
mvn -Dtest=OmsProviderVerificationTest test
```

If every contract is satisfied, all the tests pass, and the results are recorded back on the broker (since `pact.verifier.publishResults` is set to `true`).

7.6 Step 5: Check If It Is Safe to Deploy

Once both sides have been verified, you can ask the Pact Broker if the consumer is safe to deploy, using the Pact command-line tool:

```
pact-broker can-i-deploy --pacticipant posConsumerProj --version <your-version> -
-to-environment production --broker-base-url http://localhost:9292
```

This will print yes or no, depending on whether a verified, matching provider version exists.

7.7 Running the Selenide Tests

The Selenide test expects a website running locally at `http://localhost:5173/catalog`. Once that website is running, you can run the test from the project root with:

```
mvn -pl posConsumerProj -Dtest=CatalogPageTest test
```

This will open a real browser window, go to the catalog page, search for "headphones", and check that results appear.

8. Summary

This project shows two different ways of testing software:

- Pact testing checks that two separate services (POS and OMS) agree on their API, without needing to run them together.
- Selenide testing checks that an actual web page behaves correctly, by opening it in a browser and interacting with it.